

Presentation

Prerequisites

1. **Command Line Interface (CLI) Tool:** CMD, Git Bash
2. **Web Browser:** Chrome, Firefox, Safari, and Edge.
3. **Git Installed:** install Git from the [official website](#).
-To check that Git is installed,

```
git -v
```

Know the different

Git is a version control system.

- **Track Changes, Collaboration, Backup, Branching**

GitHub is a web-based platform that serves as a hosting service for Git repositories.-

- **Repository hosting, Collaboration tools, Project management, CI/CD, Community and open source**

How to use Git

Basic Git commands

1. **git config** : set configuration, set are your username and email

How to use git config

- **How to set Git username:**

```
git config --global user.name "Your Name"
```

- **How to set Git email:**

```
git config --global user.email "your.email@example.com"
```

2. **git add** : to add changes in your working directory to the staging area, Helps in Tracking

How to use git add

- **How to add a specific file:**

```
git add filename
```

- **How to add all changes in the current directory:**

```
git add .
```

- **How to add all changes in the entire project:**

```
git add -A
```

How to undo git add

remove it from the staging area

```
git rm --cached filename
```

3. **git status** : display the state of the working directory and the staging area.

- Staged changes
- Unstaged changes
- Untracked files

How to use git status

```
git status

----
## Git will Display Soething like
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
(use "git reset HEAD <file>..." to unstage)
modified:   file1.txt
new file:   file2.txt

Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified:   file3.txt

Untracked files:
(use "git add <file>..." to include in what will be committed)
file4.txt
```

4. **git commit** : to save your changes to the local repository, snapshot of the changes

How to use git commit

- **How to commit with a message:**

```
git commit -m "Your commit message"
```

Tips for commit

- **Keep it short and sweet : be brief yet descriptive.**
- **Explain why, not just what :** If the changes involve a file, explanation of why the changes were made

5. **git log** : used to display the commit history of a repository. Showing

- Commit Hash: e.g `1a2b3c4d5e6f7g8h9i0jklmnopqrstuvwxyz`
- Author : name and email, person who made the commit
- Date
- Commit Message

How to use git log

```
git log
```

- **How to limit the number of commits shown:**

```
git log -n 5
```

- **How to show the history of a specific file:**

```
git log filename
```

- **How to display graphical representation of the commit history:**

```
git log --graph --oneline
```

6. **git diff** : want to see changes in your code
- modified, added, or deleted

How to use `git diff`

- **How to see changes in the working directory that haven't been staged for commit**

```
git diff
```

- **How to see changes between the staging area and the last commit:**

```
git diff --staged
```

- **How to compare two different commits:**

```
git diff commit1_id commit2_id
```

7. **git revert** : used to create a new commit that undoes the changes made by a previous commit.

How to use git revert

```
git revert <commit-hash>
```

git reset : undo commits by moving the current branch to a specified commit

(more powerful and potentially more dangerous)

- **Data Loss:** The `--hard`

- **History Rewriting:**

How to use git reset

```
git reset --TYPE <commit-hash>
```

-- Type

- **--soft** : undoes commits but keeps your changes in the staging area.

```
git reset --soft <commit-hash>
```

- **--mixed** : undoes commits and unstages the changes.

```
git reset --mixed <commit-hash>
```

- **--hard** : completely undoes commits and discards all changes in the working directory and staging area.

```
git reset --hard <commit-hash>
```

8. **git branch** : manage branches in your repository (work on different features, bug fixes, or experiments independently from the main codebase)

How to use git branch

- **How to list all branches:**

```
git branch
```

- **How to create a new branch:**

```
git branch branch-name
```

- **How to delete a branch:**

```
git branch -d branch-name
```

- **List remote branches:**

```
git branch -r
```

- **List all branches (local and remote):**

```
git branch -a
```

git checkout : To switch between branches or restore working directory files.

- **How to switch to an existing branch:**

```
git checkout branch-name
```

9. git stash : temporarily save changes in your working directory that you are not ready to commit.

How to use git stash

- **How to stash changes:**

```
git stash
```

- **How to list stashed changes:**

```
git stash list
```

- **How to apply the most recent stash:**

```
git stash apply
```

- **How to drop a specific stash:**

```
git stash drop stash@{n}
```

- **How to clear all stashes:**

```
git stash clear
```

10. git merge: combine changes from one branch into another. (Suggesting Only when Conflict Occur)

How to use git merge

```
git merge branch-name
```

How to resolve merge conflicts

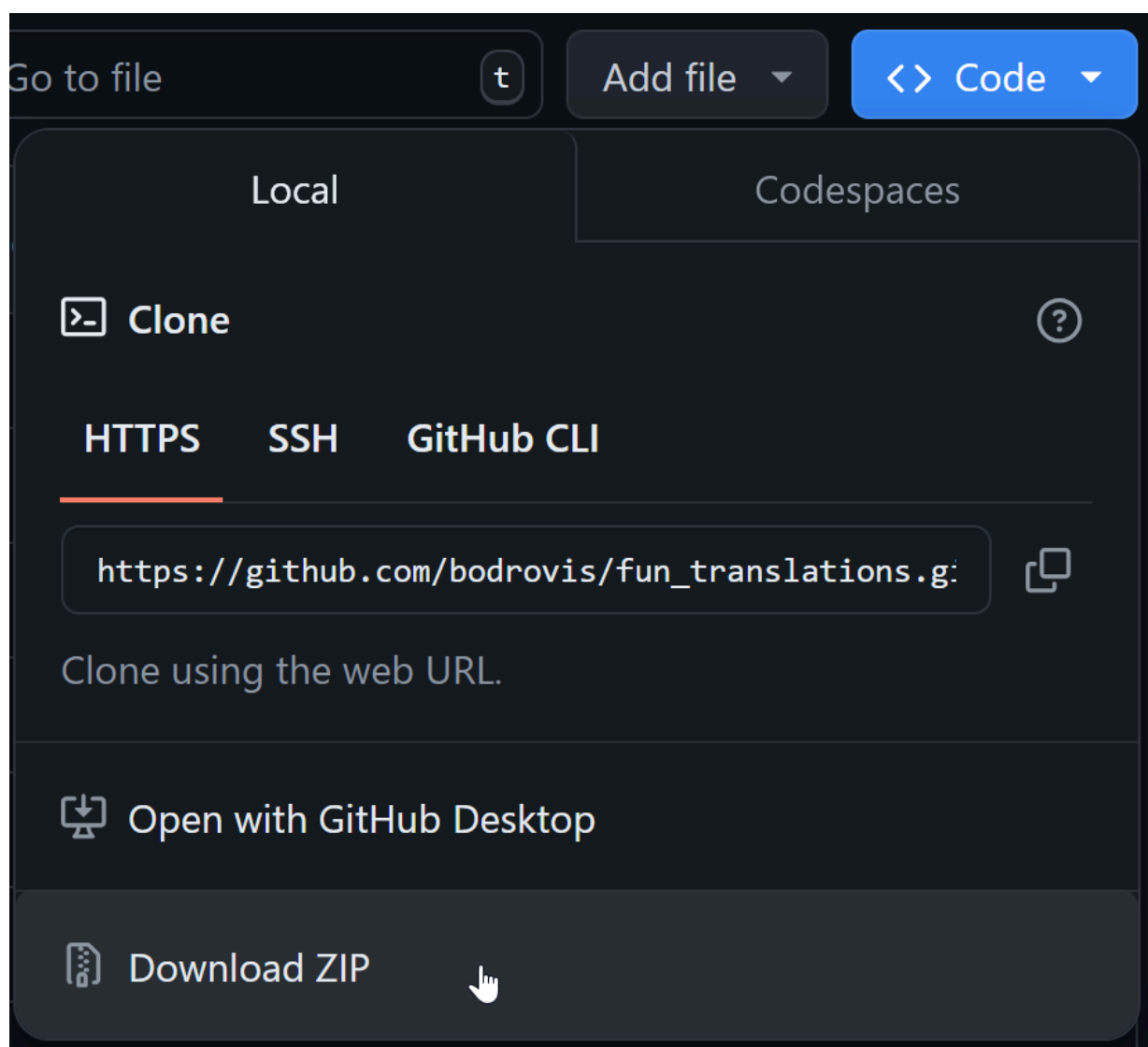
1. Pull form main branch `git pull origin`

2. Git will mark the conflicted areas in the files.
3. Open each conflicted file and manually resolve the conflicts.
4. Stage the resolved files using `git add`.
5. Complete the merge with `git commit`
6. Push to Branch

How to use GitHub

Registering on GitHub : github.com, Registration is free

1. **How to download files from GitHub**
 - GitHub repository : click "Code > Download ZIP"



2. **Connecting GitHub with local Git**
 - To allows you to push changes from local machine to remote repository
 - **Authenticating with GitHub**
 - On first push or pull from a GitHub repository
 - Authenticate via web browser

```
git push origin main
```

3. **How to clone a Git repository: git clone : create a copy of an existing Git repository**

How to use git clone (Just make sure you in the right DIR)

- **How to clone a repository:**

```
git clone <repository-url>
```

e.g

```
git clone https://github.com/username/repository.git
```

4. **git fetch** : retrieves the changes and does not merge them, 2 separate commands.

How to use git fetch

```
git fetch <remote>
```

e.g

```
git fetch origin
```

then you will want to Merge

```
git merge origin/main
```

• How to fetch all remotes:

```
git fetch --all
```

5. **git pull** : combines the functionality of `git fetch` and `git merge` ,

How to use git pull

- How to pull changes from a remote repository

```
git pull <remote> <branch>
```

e.g

```
git checkout main
```

```
git pull origin main
```

6. **git push** : upload your local repository changes to a remote repository.

How to use git push

- How to push changes to a remote repository

```
git push <remote> <branch>
```

e.g

```
git push origin <branch>
```

How to resolve push conflicts

1. Git will reject the push and display an error message.
2. Pull the latest changes from the remote repository:

```
git pull origin main
```

3. Resolve any conflicts that arise.
4. Commit the resolved changes:


```
git commit -m "Resolved conflicts"
```

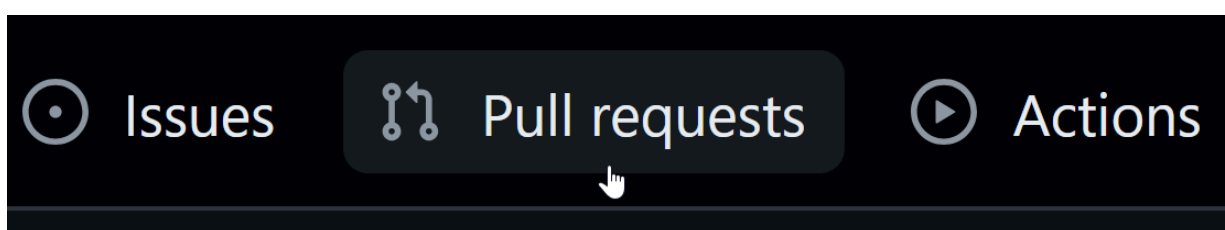
5. Push the changes again:

```
git push origin main
```

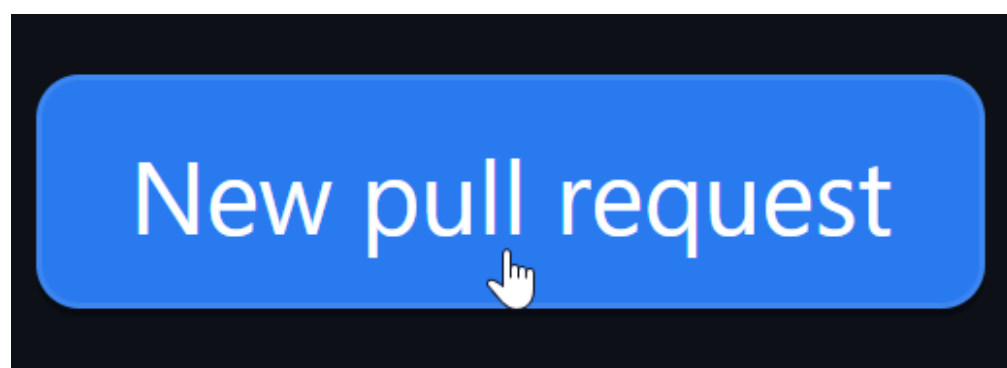
How to contribute on GitHub with pull requests

How to create a pull request

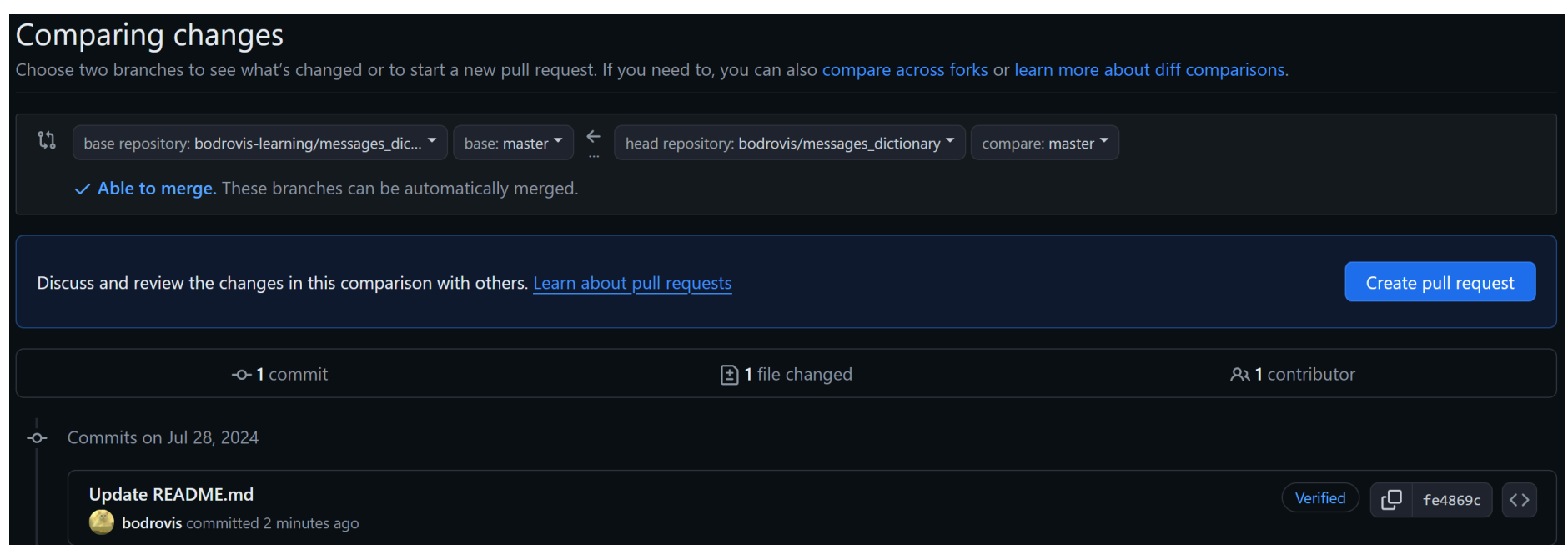
1. **Make changes in your forked repository:** Use Git commands like `git add`, `git commit`, and `git push` to make and push changes to your forked repository.
2. **Create a pull request:**
 - Navigate to the repository on GitHub.
 - Click on the "Pull requests" tab:



- Click the "New pull request" button:



- Select the branch of your repository that contains the changes. You might also need to click "compare":



3. Submit the pull request:

- Click the "Create pull request" button.
- Write a descriptive title and description for your pull request:

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff con](#)



base repository: bodrovis-learning/messages_dic... ▾

base: master ▾

← ...

head repository: bodrovis/messages_dictionary ▾

compare: master ▾

✓ **Able to merge.** These branches can be automatically merged.



Add a title

Update README.md

Add a description

WritePreview

H B I ≡ <> 🔗 ≡ ≡ ≡ ≡ 📎 @ ↻ ↩

Summary

Workflow

- **git clone:** You clone your forked repository to your local machine.
- **git add/commit/push:** You make changes locally and push them to your forked repository on GitHub.
- **Pull request:** You propose changes from your forked repository to the original repository.

Presentation

8